

AprilTag Control Car Action Group (No-PTZ)

Experiment Objective

Use the fixed camera to recognize `tag36h11` family AprilTag machine codes, mapping IDs 1–4 to chassis velocity commands to control the No-PTZ car to move forward, backward, and turn.

This program does not publish preset action group names; instead, it continuously publishes `geometry_msgs/Twist` to `/cmd_vel` based on the currently recognized machine code.

Experiment Principle

1. Control Flow

```
ESP32 camera
-> /camera/image_raw
-> Image conversion and scaling to 320x240
-> dt_apriltags AprilTag recognition
-> Filter IDs 1, 2, 3, 4, and select the smallest ID
-> Machine code mapped to Twist
-> /cmd_vel
-> Car chassis
```

The node uses two timers: the image processing timer recognizes machine codes at 15 Hz by default, and the control publish timer publishes velocity at 20 Hz by default. When no valid machine code is detected, it publishes an all-zero `Twist` to stop the car.

2. Topics and QoS

Direction	Topic	Message Type	QoS	Description
Subscribe	<code>/camera/image_raw</code>	<code>sensor_msgs/Image</code>	BEST_EFFORT, KEEP_LAST, depth=1	ESP32 camera raw image
Publish	<code>/cmd_vel</code>	<code>geometry_msgs/Twist</code>	BEST_EFFORT, VOLATILE, KEEP_LAST, depth=1	Chassis velocity command

3. Machine Code and Action Mapping

The default linear velocity is `0.2 m/s`, and the default angular velocity is `0.8 rad/s`.

AprilTag ID	linear.x	angular.z	Action
1	+linear_speed	0	Forward
2	-linear_speed	0	Backward
3	+linear_speed	-angular_speed	Forward and turn right
4	+linear_speed	+angular_speed	Forward and turn left

When multiple valid IDs appear in the frame at the same time, the program selects the ID with the smallest value. For example, when both 2 and 4 are recognized, it executes the action of ID 2. IDs outside the 1–4 range do not participate in control.

Experiment Steps

1. Hardware and Software Preparation

Item	Requirement
Car version	No-PTZ version
Camera	ESP32 camera, fixed facing the front of the car
AprilTag	tag36h11 family, prepare IDs 1, 2, 3, 4
Python dependencies	dt-apriltags, OpenCV, NumPy

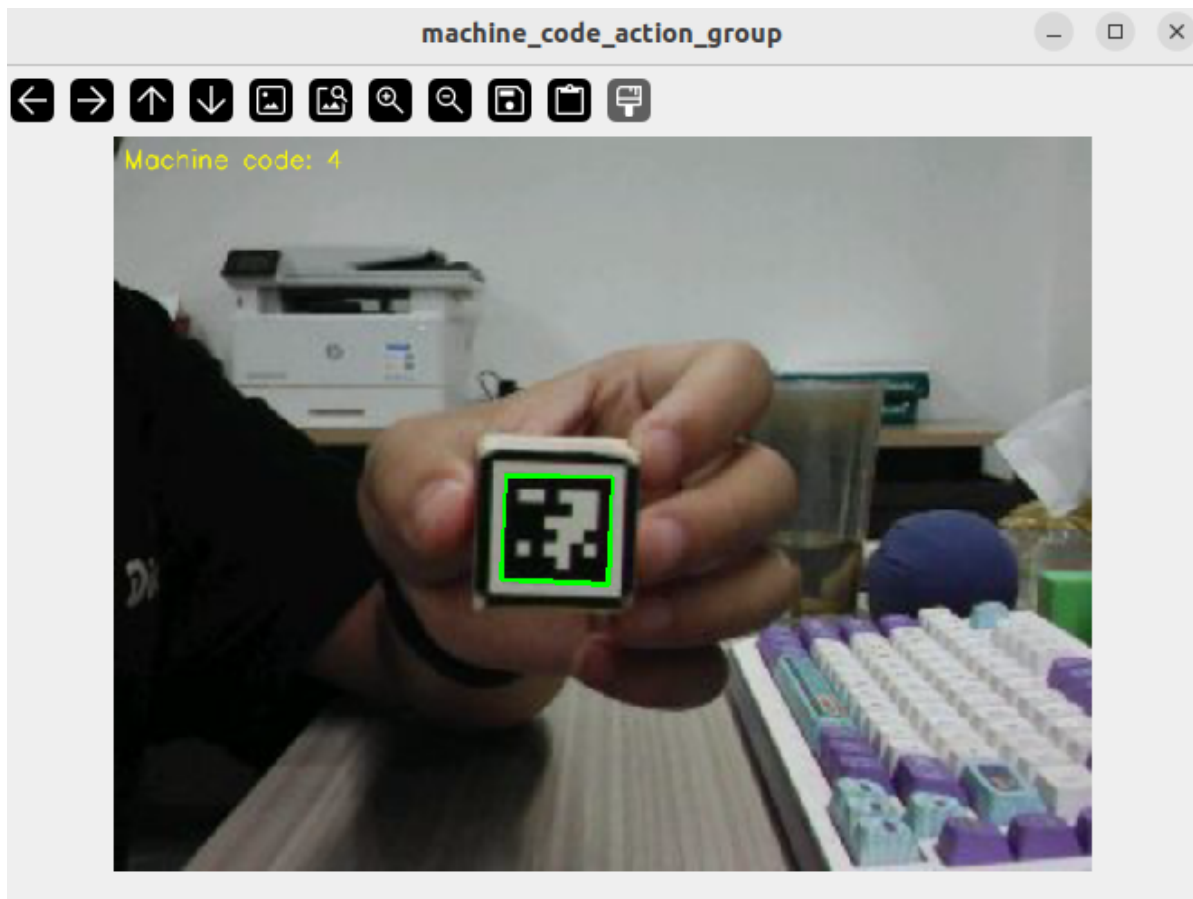
2. Start the Chassis, Agent, and Camera (Terminal 1)

```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-10-47-55-372666-yahboom-149737
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [149778]
[INFO] [camera_driver-2]: process started with pid [149780]
[INFO] [robot_state_publisher-3]: process started with pid [149782]
[INFO] [joint_state_publisher-4]: process started with pid [149784]
[INFO] [ekf_node-5]: process started with pid [149803]
[camera_driver-2] [INFO] [1785206876.886150022] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[joint_state_publisher-4] [INFO] [1785206877.007751174] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description topic.
[micro_ros_agent-1] [1785206877.173080] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785206877.754391885] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785206877.754890084] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785206877.754912957] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754918130] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785206877.754924428] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785206877.754929199] [robot_state_publisher]: got segment can3_Link
[robot_state_publisher-3] [INFO] [1785206877.754935071] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785206877.754939792] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785206877.754944035] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785206877.754949951] [robot_state_publisher]: got segment rfwheel_Link
[camera_driver-2] [WARN] [1785206879.871708745] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785206880.046230399] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start AprilTag Machine Code Control (Terminal 2)

```
ros2 run microros_v2_no_ptz_visual_control_pkg
apriltag_action_group_control_node
```



4. Stop the Program

In the OpenCV window, press lowercase `q` or `Esc` to exit. Before the node is destroyed, it publishes one all-zero `Twist`; pressing `Ctrl+C` directly also executes the stop/cleanup logic.

Experiment Result

- The debug window shows the currently recognized machine code, e.g., `Machine code: 1`.
- The car moves forward when ID 1 is recognized, and backward when ID 2 is recognized.
- The car moves forward and turns right when ID 3 is recognized, and forward and turns left when ID 4 is recognized.
- When the tag is removed, another ID is recognized, or no image is available yet, the control output is zero velocity and the car stops.

Code Explanation

Source:

```
microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/apriltag_action_group_control_node.py
```

Launch file:

```
microros_v2_no_ptz_visual_control_pkg/launch/machine_code_action_group_control.launch.py
```

Note: The current launch file and `setup.py` still reference `machine_code_action_group_control_node`, but the corresponding `machine_code_action_group_control_node.py` does not exist in the source directory and has not been synchronized with the `apriltag_action_group_control_node.py` used by this tutorial. Therefore, this section starts the node using the Python module command above instead of that launch file.

1. Node Initialization

The node is named `apriltag_action_group_control_node`, and its main initialization is as follows:

```
super().__init__("machine_code_action_group_control_node")

self.declare_parameter("image_topic", "/camera/image_raw")
self.declare_parameter("cmd_vel_topic", "/cmd_vel")
self.declare_parameter("tag_family", "tag36h11")
self.declare_parameter("quad_decimate", 2.0)
self.declare_parameter("processing_width", 320)
self.declare_parameter("processing_height", 240)
self.declare_parameter("image_processing_rate_hz", 15.0)
self.declare_parameter("control_rate_hz", 20.0)
self.declare_parameter("linear_speed", 0.2)
self.declare_parameter("angular_speed", 0.8)
self.declare_parameter("show_debug_window", True)
self.declare_parameter("window_name", "machine_code_action_group")
```

The program uses `dt_apriltags.Detector` to create the detector, and configures both the image subscription and the `/cmd_vel` publisher with BEST_EFFORT, KEEP_LAST, depth=1 to match the camera and chassis communication links.

2. AprilTag Recognition

During each image processing cycle, the program first scales the latest image to `processing_width` × `processing_height`, then converts it to grayscale and calls the detector:

```
gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)
tags = self._detector.detect(gray, estimate_tag_pose=False)
valid_ids = [
    int(tag.tag_id) for tag in tags
    if int(tag.tag_id) in (1, 2, 3, 4)
]
code = min(valid_ids) if valid_ids else None
```

Here `estimate_tag_pose=False` means the node only needs the tag ID and corner points, and does not compute distance, position, or pose. Valid recognized tags are drawn with a green border in the debug window, and the current machine code is displayed.

3. Mapping Machine Codes to Twist

The control timer reads the most recent recognition result and publishes a `Twist` in each callback:

```

cmd = Twist()
if code == 1:
    cmd.linear.x = self._linear_speed
elif code == 2:
    cmd.linear.x = -self._linear_speed
elif code == 3:
    cmd.linear.x = self._linear_speed
    cmd.angular.z = -self._angular_speed
elif code == 4:
    cmd.linear.x = self._linear_speed
    cmd.angular.z = self._angular_speed
self._cmd_pub.publish(cmd)

```

`Twist()` defaults to zero, so a stop command is published when `code` is `None` or any other value. This program performs continuous velocity control, not a discrete action group that publishes once and waits for the chassis to execute.

4. Key Parameters

Parameter	Type	Default	Description
<code>image_topic</code>	string	<code>/camera/image_raw</code>	Input image topic
<code>cmd_vel_topic</code>	string	<code>/cmd_vel</code>	Chassis velocity topic
<code>tag_family</code>	string	<code>tag36h11</code>	AprilTag family
<code>quad_decimate</code>	float	<code>2.0</code>	Detection decimation factor; larger is usually faster but small tags may be harder to recognize
<code>processing_width</code>	int	<code>320</code>	Processing image width
<code>processing_height</code>	int	<code>240</code>	Processing image height
<code>image_processing_rate_hz</code>	float	<code>15.0</code>	Image processing rate
<code>control_rate_hz</code>	float	<code>20.0</code>	<code>/cmd_vel</code> publish rate
<code>linear_speed</code>	float	<code>0.2</code>	Forward/backward linear velocity; the program takes the absolute value
<code>angular_speed</code>	float	<code>0.8</code>	Turning angular velocity; the program takes the absolute value
<code>show_debug_window</code>	bool	<code>true</code>	Whether to show the OpenCV debug window
<code>window_name</code>	string	<code>machine_code_action_group</code>	Debug window name